Completed development work

llama.cpp Router is a Windows desktop application for starting and controlling local language-model inference. It wraps the llama.cpp server in a native window, keeps model and launch settings together, and is being developed into a wider runtime manager and request router.

> **Built and tested does not mean released.** The tested development update is in draft PR #5. Its implemented features have not been merged into main or established as a finished production release. A portable update exists; a complete fresh-install release is still pending.

How the app works

1. **Choose the model and settings.** Use local GGUF files, choose model folders, and set the server's GPU placement, context, cache and related launch options. The development update adds the Models-page profile editor and contextual guides.
2. **Resolve hardware and validate the request.** The wrapper discovers runtime devices, resolves saved physical GPU UUIDs to the current device order, and rejects incompatible or malformed settings before startup.
3. **Start a managed local server.** The desktop shell starts llama-server with an argument array and its own child environment, then checks readiness. The runtime loads model weights and performs inference; the wrapper controls startup and settings.
4. **Use chat or connect an API client.** The web UI appears inside the desktop window. API clients can connect to the running llama.cpp server's compatible endpoints. The Windows Tauri build uses localhost port 18080; other launch modes may differ.
5. **Inspect results and change deliberately.** Read status and logs, inspect the saved launch plan, and use hover help to understand controls. Saving launch settings does not move an already loaded model; restart the app to apply those changes.

145 Node tests	14 native tests	Dual-3090 inference checked

The real inference checks used a small Qwen2.5 0.5B model, 2048 context and one sequence. They establish correctness for that configuration, not large-model capacity or general performance.

Source: draft PR #5 and its PROJECT_STATUS.md; source commit 973b2e3. Snapshot checked November 6, 2026.

C01 Windows desktop launch and update build

Delivered. The Windows shell and companion manager compile, and a portable update package contains the updated binaries, UI, source snapshot and validation records.

How it works. The Tauri shell opens the local web UI in a native desktop window and manages a llama.cpp server. The repository also retains a Node launcher that can use Edge or Chrome as an app window.

Evidence. Windows release binaries and the production UI build completed successfully.

Limits. This is an update for an existing portable installation. It does not establish that a complete new-machine installer is finished.

C02 Model, log and engine-control workspace

Delivered. The development UI includes local model-folder, model-list, cache/log-folder and log-viewing controls. Separate pages expose management controls for WSL vLLM, TensorRT-LLM and llama-bench.

How it works. Model folders can be selected without moving the GGUF files. Engine services have their own configuration and endpoints; they remain separate from the llama.cpp chat server.

Evidence. The controls are implemented in the submitted source and included in the built UI.

Limits. Control availability is not a claim that every WSL engine has completed real model inference testing. Full service validation remains planned.

C03 GPU discovery with a CUDA identity fallback

Delivered. The runtime discovers the available GPUs on the validation machine: two RTX 3090s and one RTX 4070.

How it works. Device probes have time and output bounds. When NVIDIA's NVML monitoring interface fails, the companion helper reads physical identities from the Windows CUDA driver.

Evidence. The release helper returned names, total memory, PCI locations and UUIDs for all three detected NVIDIA cards.

Limits. Missing monitoring readings remain unknown. Detection on this machine does not certify every AMD, Intel or other backend.

C04 Durable, ordered GPU selection

Delivered. A profile can save an ordered set of physical GPU UUIDs rather than relying on CUDA index numbers that may change.

How it works. The wrapper resolves identities at launch, preserves the requested order and refuses to replace a missing card silently. A conflicting CUDA visibility override produces a diagnostic.

Evidence. A real launch with the two 3090 UUIDs in reversed order used the intended physical cards and returned a valid streamed response.

Limits. UUID selection chooses devices for one managed server. It does not create one independent model worker per GPU.

C05 Real dual-3090 layer-split inference

Delivered. One model was loaded across both RTX 3090s using layer splitting and explicit allocation weights.

How it works. llama.cpp performs the actual tensor allocation and inference. The wrapper emits the selected device list, split mode and allocation ratios.

Evidence. Qwen2.5 0.5B Instruct Q4_K_M returned READY. Logs show 25/25 layers offloaded, with model and KV buffers on both cards, at 2048 context and one sequence.

Limits. This is a small-model correctness result. Large-model capacity, tensor mode, mixed-card performance and multi-worker concurrency are not proven by this test.

C06 Versioned, durable launch profiles

Delivered. Version 1 profiles preserve settings and extension data, track revisions and protect against stale saves.

How it works. Legacy flat settings are read without rewriting the source file. A save validates first, obtains a lock and replaces the settings file atomically.

Evidence. Node and Rust tests cover migration, persistence, invalid-save preservation and stale-revision rejection.

Limits. The current editor manages the active whole-server launch profile. A complete deployment/profile manager for multiple workers is still to come.

C07 Consistent launch validation in Node and Rust

Delivered. Both launch compilers check numeric ranges, Boolean values, GPU split/cache combinations, quoting and managed option conflicts.

How it works. Shared fixtures keep the entry points aligned. Accepted numeric text is emitted canonically; explicit context and requested floating-point V-cache formats are retained.

Evidence. The wrapper's 145 Node tests and 14 native tests include the shared launch contract and regression cases.

Limits. Static validation checks the request. It cannot prove that every model fits or that an experimental mode works on a particular binary.

C08 Protected child environment and redacted diagnostics

Delivered. Managed environment variables and extra arguments cannot bypass the validated settings, and credential values are redacted from launch diagnostics.

How it works. Discovery and launch use the resolved child environment, including the managed cache path. Inherited P2P is removed by default; global GPU configuration is not rewritten for one launch.

Evidence. Regression tests cover environment normalization, managed overrides, cache consistency and secret redaction.

Limits. This is launch-envelope protection, not a claim of a comprehensive security audit or completed multi-provider credential management.

C09 Launch editor and requested/effective preview

Delivered. The Models page contains a collapsible launch-profile editor, physical GPU selector, saved-launch preview and runtime-inspection controls.

How it works. Users can inspect context, layers, split weights, cache choices, Flash Attention and expert arguments before restarting the server.

Evidence. Svelte checking reported zero errors and warnings. Browser checks exercised profile saves, revision refresh, preview, unavailable inventory and narrow layouts.

Limits. Launch changes require an app restart. The controls currently configure the whole server, not individual model instances.

C10 Global hover help and input guidance

Delivered. Actions and inputs can reveal a circled question button, compact help, and a nearly full-window documentation panel with black translucent styling and background blur.

How it works. Authored topics explain technical launch inputs, suggested starting values, trade-offs, usage and troubleshooting. Help opens independently of the underlying action.

Evidence. Browser checks cover hover behavior, action isolation, F1/Escape, focus, dynamic controls, secret redaction and narrow windows.

Limits. Suggested values are examples for comparison. They are never applied automatically and do not replace a model's own configuration requirements.

C11 Package identity and bounded capability inspection

Delivered. The wrapper fingerprints the installed executables/DLLs and captures bounded version/help evidence for the selected runtime.

How it works. Startup checks required advertised flags. A feature being listed in help is recorded separately from successful inference for an exact model/build combination.

Evidence. Tests cover changed package/driver identity, unsupported arguments, timeouts and excessive probe output.

Limits. This does not implement automatic downloads, live package replacement, compatibility promotion or rollback.

C12 Streaming timing and reproducible correctness checks

Delivered. A reusable canary records runtime/model hashes, resolved settings, allocation logs, generated output, first-token timing and available token counts.

How it works. TTFT starts at request submission and ends on the first nonempty generated content, reasoning or tool delta. Role-only and empty events do not count; content timing is separate.

Evidence. Both direct-device and UUID-ordered dual-3090 streaming canaries passed. Node tests verify timing semantics; browser checks and production builds also passed.

Limits. Timing is currently a test-runner capability. A live monitoring dashboard, repeated benchmarks and production performance guarantees are not finished.